Gul Bil projektrapport

Christian Munch Hammervig
Nikolaj Valbjørn Have

Indhold

Produktrapport 1
Linux Maskine 1
API 2
Systemarkitektur 3
Datamodeller 3
Endpoints 4
Frontend 6
Teknologier 6
NextJS 9
Arduino Integration 10
Materialer og reference 11

Produktrapport

Der skal laves en Hjemmeside og en API til at håndtere spil med gule biler.

Linux Maskine

Løsning til vores maskine til tests.

Da dette kun en virtuel maskine til at kunne teste deployment, har vi valgt ikke at lave nye brugere på den.

Og da det kun er for administration, er der ikke oprettet ssh nøgler og den kræver bare et password for at logge ind.

Alt kører gennem Docker, og Portainer. Det tillader håndtering af brugere og tilladelser gennem et Web UI.

API

Løsning til alt backend.

Krav:

- Håndtere kommunikation med Database og S3 Storage Bucket.
- Håndtere registrering og login af brugere.
- Skal kunne beskære user avatars og uploade dem til en S3 storage bucket.
- Skal kunne oprette og håndtere spil, som klienter skal kunne forbinde til med SignalR.

Autentificering:

Den skal kunne håndtere

- Register
- Login
- Refresh
- Avatar upload
- Avatar get

Gul bil spillet:

Den skal kunne oprette og starte spil.

Man skal kunne forbinde til et spil, ved hjælp at den kode som spillet

Spillende skal foregå server siddet, for at være fælles for alle og kunne synkronisere alle handlinger mellem spillere.

Det skal bygges med SignalR i stedet for websockets, da det er fuldt integreret i APS .NET Core.

Systemarkitektur

Skrevet i C#

- .NET 10
- ASP.NET Core
- Projektstruktur (Controllers, Services, Repositories)
- Bruger EFCore til styring af databasen.
- Bruger MinIO til håndtering af brugeres avatars.

Datamodeller

- Domænemodeller består af Common, User og RefreshToken.
- Alle nedarver fra common for at lave en nem fælles struktur.
  Som indeholder deres (Id, CreatedAt og UpdatedAt)

Eksempel på User modellen:

```
public class Common
{
    12 references
    public Guid Id { get; set; }
    1 reference
    public DateTime CreatedAt { get; set; }
    1 reference
    public DateTime UpdatedAt { get; set; }
}
```

```
public class User : Common
{
    8 references
    public string Username { get; set; }
    5 references
    public string HashedPassword { get; set; }

    2 references
    public bool IsDisabled { get; set; } = false;

    6 references
    public Roles Role { get; set; } = Roles.User;
}
```

Endpoints

AuthController

- Login
  - Logger en bruger ind of afleverer en jwt-token, expiry i sekunder og en refresh token.
  - HTTP-Metode: Post
  - Url: /Auth
  - Body:
    - Username
    - Password
- Refresh
  - Afleverer en ny jwt-token og en ny refresh token til at kunne forblive logget ind.
  - HTTP-Metode: Post
  - Url: /Auth/Refresh
  - Body:
    - RefreshToken
- Register
  - Tillader registrering af en ny bruger
  - http-metode: Post
  - url: /Auth/Register
  - Body:
    - Username
    - Password
    - ConfirmPassword
- UserInfo
  - Afleverer alt information om den bruger som er logget ind.
  - http-metode: Get
  - Url: /Auth
- ChangePassword
  - Tillader skift af password.
  - http-metode: Post
  - Url: /Auth/ChangePassword
    - Password
    - ConfirmPassword

- UploadAvatar
    - Tillader en bruger at sætte sit profilbillede.
    - http-metode: Post
    - Url: /Auth/ Avatar
    - Body:
      Form med en PNG.
- GetAvatar
    - Tillader at finde en brugers avatar.
    - http-metode: Get
    - Urls:
        - /Auth/Avatar (Giver billedet til den nuværende bruger.)
        - /Auth/Avatar/[UserId] (Afleverer et billede for en specifik bruger.)
- RemoveAvatar
    - Tillader fjernelse af ens avatar.
    - http-metode: Delete
    - Urls:
        - /Auth/Avatar (Fjerne billedet fra den nuværende bruger.)
        - /Auth/Avatar/[UserId]
          (Tillader en administrator at fjerne en specifik brugers avatar.)

GameController

- StartGame
    - Starter et nyt spil, og aflevere kode til det, som man kan bruge til at forbinde til det.
    - http-metode: Post
    - Url: /Game/Start
- ListGames
    - Afleverer en liste af alle startede spil og info om dem (Antal spillere og Stage)
    - http-metode: Get
    - Url: /Game
- StopGame
    - Tillader en admin at stoppe et spil.
    - http-metode: Delete
    - Url: /Game/[Id]

Frontend

Teknologier

Skrevet af Christian

Framework

NextJS

Pakker

- Udsnit fra package.json

```
"@emotion/css": "^11.13.5",
"@mui/material": "^7.3.9",
"@microsoft/signalr": "^10.0.0",
"axios": "^1.13.6",
"cropperjs": "1.6.2",
"crypto-js": "^4.2.0",
"react-hook-form": "^7.72.0",
"react-icons": "^5.6.0",
"react-loading-indicators": "^1.0.1",
"yup": "^1.7.1"
```

Krav

- Håndtere login og data fra API
- Håndtere brugers data på en sikker måde
- Håndtere at oprette og joine et spil
- Håndtere brugers avatar

Pakke beskrivelse

- **MUI - @emotion/css**
  - Jeg har valgt MUI da det er bygget til React og det er ikke en masse forkortelser som ingen kan huske, men er bygget som css bare med camelcase i stedet for bindestreg. De har også egne komponenter med en prop der hedder sx til styling.

```
<Box
    component="img"
    src={avatar}
    className="avatar-img"
    sx={{
        width: 120,
        height: 120,
        borderRadius: "50%"
    }}
/>
```

Foroven ses en MUI Box, den kan bruges som div kan i HTML men så det ses her kan den også bruges som andre komponenter og her ses at man kan kombinere det med almindelig css.

- **@microsoft/signalr**
  - Denne pakke bruges til at forbinde med serveren, den del der ikke er en API og da vores server er skrevet i C# så var det bedste valg denne pakke da den også kan bruges til Javascript.

- **cropperjs**
  - Denne har vi valgt for at så man kan vælge f.eks. et lille område på et billede uden at skulle sende hele billedet eller selv redigere billedet.

- **react-icons**
  - I denne pakke kan man finde flere forskellige ikoner.

- **react-loading-indicators**
  - Ved flere tilfælde kan der risikeres at det tager lidt tid at hente informationer som at hente det krypterede data fra localstorage og for at brugeren ikke skal tro at der ikke sker noget og/eller bliver utålmodig så bruges denne til at vise at der sker noget, men i visse tilfælde er det kun i kort tid.

- **crypto.js**
  - Når en bruger logger ind, gemmes data i localstorage og for at være sikre så bliver data krypteret med denne pakke.

```
const SECRET_KEY = process.env.NEXT_PUBLIC_STORAGE_KEY;

// Gem krypteret data i localStorage
export const gemKrypteret = (key, data) => {
    try {
        if (!SECRET_KEY) {
            localStorage.setItem(key, JSON.stringify(data));
            return;
        }
        const krypteret = CryptoJS.AES.encrypt(JSON.stringify(data), SECRET_KEY).toString();
        localStorage.setItem(key, krypteret);
    } catch (err) {
        console.error("Fejl ved kryptering og lagring:", err);
    }
};
```

- **Axios**
  - Denne pakke bruges i stedet Javascripts's fetch til at kunne sende JWT token videre uden at skulle finde den hver gang.

```
axiosInstance.interceptors.response.use(
    (res) => res,
    async (error) => {
        const originalRequest = error.config;

        const isAuthRoute =
            originalRequest.url.includes('/api/userdata/login') ||
            originalRequest.url.includes('/api/userdata/refresh');

        if (
            error.response?.status === 401 &&
            !originalRequest._retry &&
            !isAuthRoute
        ) {
            originalRequest._retry = true;
            try {
                await refresh();

                // 🔥 HENT NYT TOKEN
                const user = laesDekrypteret('jwt');
                if (user?.jwtToken) {
                    originalRequest.headers.Authorization = `Bearer ${user.jwtToken}`;
                }

                return axiosInstance(originalRequest);
            } catch (err) {
                console.error('Refresh fejlede:', err);
                return Promise.reject(err);
            }
        }

        return Promise.reject(error);
    }
);
```

- **react-hook-form og yup**
    - Disse 2 pakker er til at lave formulerer og at validere dem. Fordelen ved at bruge react-hook-form er at så bliver der ikke lavet et render af siden for hver gang man skriver et bogstav.
      react-hook-form har sin egen validering, men der er mulighed for at lave sin egen eller bruge en de validerings pakker der findes og yup er nem at bruge og der mange muligheder

```
const defaultValues = {
    username: '',
    password: '',
    confirmPassword: '',
}

const schema = Yup.object().shape({
    username: Yup.string().transform((value) => value.trim()).required(usernameFejl),
    password: Yup.string()
        .transform((value) => value.trim())
        .required(passwordFejl)
        .min(8, 'Your password must be at least 8 characters long')
        .matches(/[0-9]/, 'There must be at least one number')
        .matches(/[A-Z]/, 'There must be at least one uppercase letter')
        .matches(/[^A-Za-z0-9]/, 'There must be at least one special character'),
    confirmPassword: Yup.string().transform((value) => value.trim())
        .required(bekraeftfejl)
        .oneOf([Yup.ref('password')], ens),
})
```

NextJS

NextJS blev valgt fordi det er nemt at arbejde med, det er et open-source React-framework. Der er både en server-side og en client-side, det vil sige at man kunne lave hele projektet i nextJS. Jeg valgte at lade client-side kalde server-side og server-side laver så et kald til vores API. På denne måde er det ikke browseren der laver kaldet til API'en men server-side på front-end og så kommer vi lidt uden om cors problemer, men vi skal dog stadigvæk låse op for cors til signalr, da den skal have direkte forbindelse til serveren for at få den hurtigst mulige forbindelse.

Arduino Integration

Til dette spil er der også lavet en start til en Arduino integration, med en MKR Wifi 1010.

- Javascript kan findes her.
- Og her kan findes et lille html dokument til test.

Dette starter en instans af klassen ArduinoClient, som håndterer al kommunikation mellem Arduino og browser, over seriel port.

- Den oprettes og kalder connect sådan her.

```
const client = new ArduinoClient(log);

document.getElementById("connectBtn").onclick = () => client.connect();
```

Klienten har en eventhandler som her kaldes til at logge eventet, i debug.html siden.

```
client.on("buttonPressed", (data) =>
  log("EVENT buttonPressed: " + JSON.stringify(data)),
);
client.on("tempUpdate", (data) =>
  log("EVENT tempUpdate: " + JSON.stringify(data)),
);
```

Og hvis led 1 skal sættes, kan denne bruges.
Den kalder client.request(”setLed”, json object)

```
document.getElementById("ledColor_1").onchange = async (e) => {
  const color = e.target.value;
  const r = parseInt(color.substr(1, 2), 16);
  const g = parseInt(color.substr(3, 2), 16);
  const b = parseInt(color.substr(5, 2), 16);

  const res = await client.request("setLED", { i: 0, r, g, b });
  log("LED RESPONSE: " + JSON.stringify(res));
};
```

Materialer og reference

Dokumentation og diagrammer kan findes på vores GitHub.
h5-projekt-h5-gruppe-4/Documents at main · Mercantec-GHC/h5-projekt-h5-gruppe-4